

멀티에이전트 마이그레이션 설계도

멀티에이전트 프로젝트

2026-05-28

CONTENTS

멀티에이전트 마이그레이션 설계도	
1. 개요	
목표	
2. 시스템 아키텍처	
3. 에이전트 역할 정의	
4. 마이그레이션 플로우	
기존 방식 (단일 에이전트)	
멀티에이전트 방식 (신규)	
5. 태스크 분해 전략	
원칙	
태스크 구성 예시 (본 프로젝트)	
6. 품질 보증 이중 검증	
단계 1: 스펙 컴플라이언스 리뷰	
단계 2: 코드 품질 리뷰	
7. 구현 결과	
최종 구성	
기술 스택	
8. 기대 효과	

멀티에이전트 마이그레이션 설계도

작성일: 2026-05-28

프로젝트: Next.js 보일러플레이트 — 멀티에이전트 기반 개발 자동화

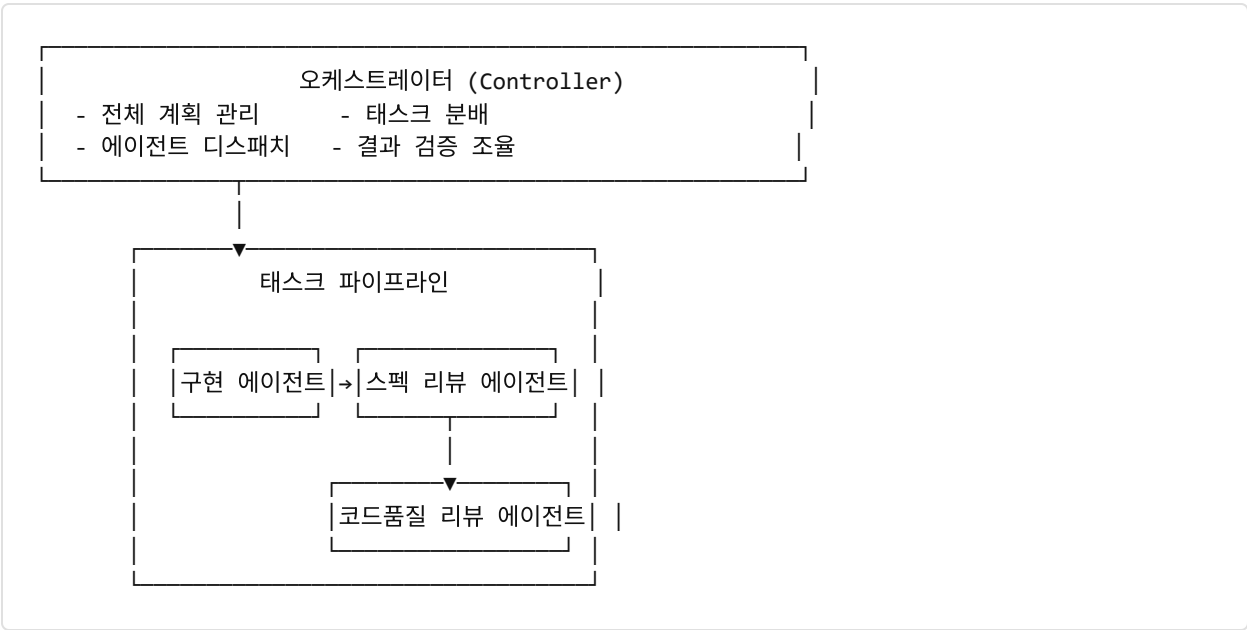
1. 개요

본 문서는 소프트웨어 개발 프로세스를 **멀티에이전트 오케스트레이션** 방식으로 전환하는 설계를 기술한다. 단일 AI 에이전트 방식의 한계를 극복하고, 역할이 분리된 복수의 에이전트가 협력하여 고품질 코드를 생산하는 구조를 정의한다.

목표

- 반복적인 보일러플레이트 작업을 자동화하여 개발자가 비즈니스 로직에 집중
- 에이전트별 역할 분리로 컨텍스트 오염 없이 독립적인 작업 수행
- 스펙 리뷰 → 코드 품질 리뷰 이중 검증으로 코드 신뢰성 확보

2. 시스템 아키텍처



3. 에이전트 역할 정의

에이전트	역할	모델
오케스트레이터	계획 로드, 태스크 순서 결정, 결과 통합	Claude Sonnet
구현 에이전트	태스크 단위 코드 작성, 자가 검토, 커밋	Claude Sonnet / Haiku
스펙 리뷰 에이전트	구현 결과와 스펙 대조 검증	Claude Haiku
코드 품질 에이전트	클린 코드, 아키텍처, 보안 검토	Claude Haiku / Sonnet
브레인스토밍 에이전트	요구사항 수집, 설계 대안 제시	Claude Sonnet
계획 에이전트	스펙 기반 구현 계획 문서화	Claude Sonnet

4. 마이그레이션 플로우

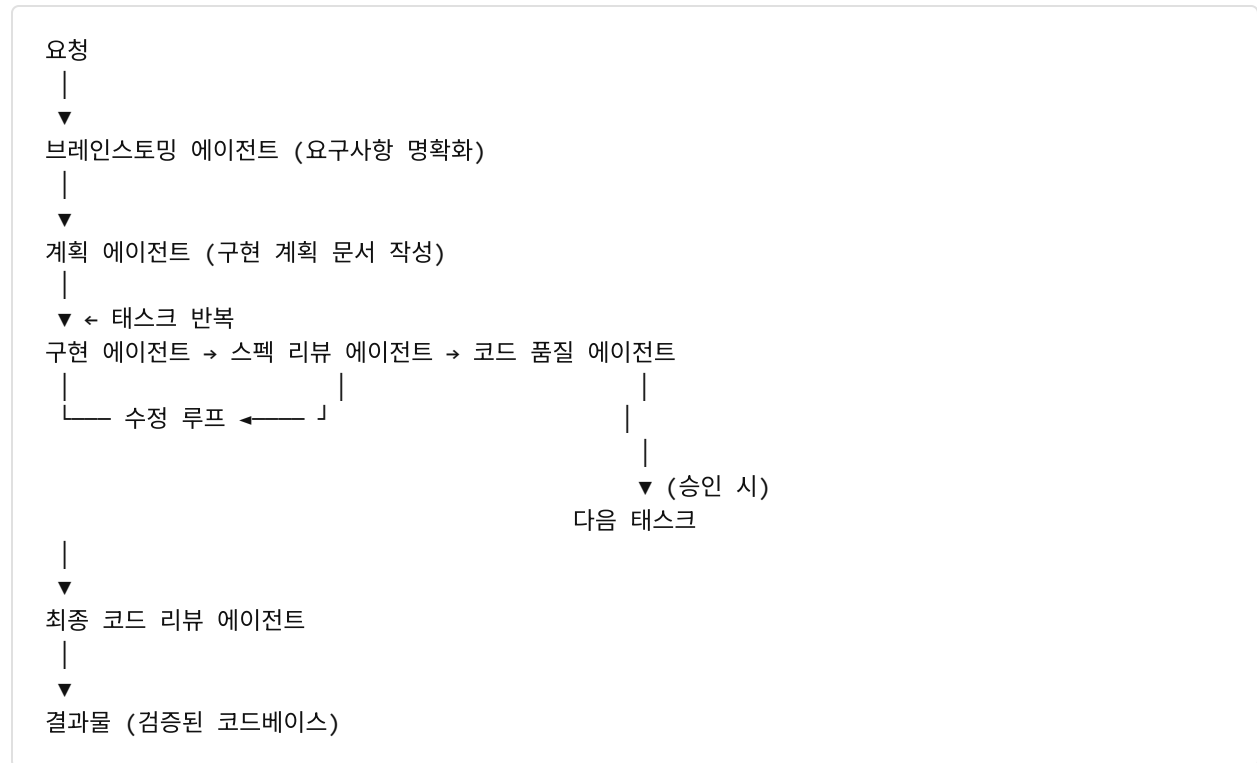
기존 방식 (단일 에이전트)

사용자 요청 → 에이전트 → (컨텍스트 오염, 일관성 저하) → 결과물

문제점:

- 긴 대화일수록 초기 컨텍스트 희석
- 검토 없이 바로 코드 생성 → 오류 누적
- 단일 장애점: 에이전트 실패 시 전체 중단

멀티에이전트 방식 (신규)



5. 태스크 분해 전략

원칙

1. **단일 책임**: 태스크 하나 = 파일 13개, 25분 단위
2. **독립성**: 태스크 간 공유 상태 최소화
3. **커밋 단위**: 각 태스크 완료 시 즉시 커밋 (롤백 가능)
4. **순차 실행**: 구현 에이전트는 병렬 실행 금지 (충돌 방지)

태스크 구성 예시 (본 프로젝트)

#	태스크	담당 파일
1	프로젝트 스캐폴딩	<code>package.json</code> , <code>next.config.ts</code>
2	의존성 설치	<code>package.json</code>
3	Prisma 스키마	<code>prisma/schema.prisma</code> , <code>lib/db.ts</code>
4	NextAuth 설정	<code>lib/auth.ts</code> , <code>app/api/auth/[...nextauth]/route.ts</code>
5	미들웨어	<code>middleware.ts</code>
6	UI 컴포넌트	<code>components/ui/*</code>
7	Providers	<code>components/providers.tsx</code>
8	Server Actions	<code>server/actions/auth.ts</code>
9	페이지 구현	<code>app/**/*.*tsx</code>
10	코드 품질 도구	<code>.prettierrc</code> , <code>.husky/*</code>
11	Seed + 환경변수	<code>prisma/seed.ts</code> , <code>.env.example</code>
12	최종 검증	DB 마이그레이션, 타입 체크

6. 품질 보증 이중 검증

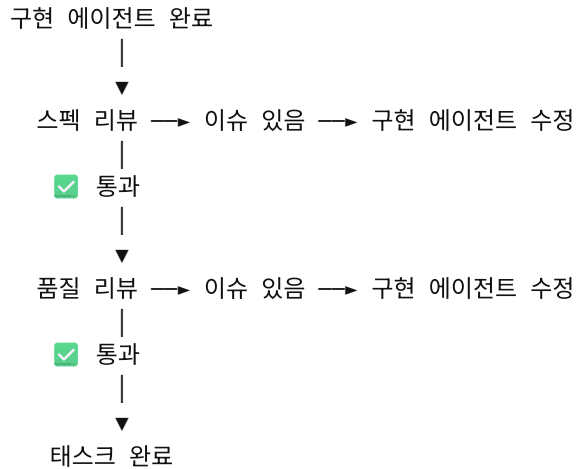
각 태스크 완료 후 두 단계 리뷰를 의무화한다.

단계 1: 스펙 컴플라이언스 리뷰

- 구현된 코드가 스펙과 정확히 일치하는지 검증
- 누락된 기능 / 과도한 구현 모두 지적
- **스펙 불일치 시:** 구현 에이전트가 재수정 → 재리뷰

단계 2: 코드 품질 리뷰

- 클린 코드, 타입 안전성, 보안, 아키텍처 검토
- Critical / Important / Minor 3단계 심각도 분류
- **Critical · Important 이슈 존재 시:** 수정 후 재리뷰



7. 구현 결과

본 설계를 적용하여 Next.js 보일러플레이트를 **12개 태스크**, **약 14회 에이전트 디스패치**로 완성하였다.

최종 구성

```

nextjs-boilerplate/
├─ app/
│  ├─ (auth)/login/      ← 로그인
│  ├─ (auth)/register/   ← 회원가입
│  ├─ (dashboard)/dashboard/ ← 보호된 대시보드
│  ├─ api/auth/[...nextauth]/ ← Auth.js 라우트
│  └─ layout.tsx
├─ components/ui/        ← shadcn/ui 컴포넌트
├─ lib/
│  ├─ auth.ts            ← NextAuth 설정
│  └─ db.ts              ← Prisma 싱글톤
├─ server/actions/auth.ts ← Server Actions
├─ prisma/schema.prisma  ← DB 스키마
└─ middleware.ts          ← 라우트 보호
  
```

기술 스택

영역	기술
프레임워크	Next.js 16 (App Router)
인증	Auth.js v5 + Credentials
DB / ORM	PostgreSQL + Prisma 7
UI	shadcn/ui + Tailwind v4
API	Server Actions
코드 품질	ESLint + Prettier + Husky

8. 기대 효과

지표	기존 방식	멀티에이전트 방식
보일러플레이트 구축 시간	2~4시간 (수동)	30분 이내 (자동화)
코드 리뷰 누락률	높음	0% (이중 검증 의무화)
에이전트 컨텍스트 오염	빈번	없음 (태스크별 격리)
재작업 발생률	중간	낮음 (스펙 기반 구현)

본 문서는 멀티에이전트 오케스트레이션 방법론 적용 사례를 기록한 설계도입니다.